

CSCB09H Worksheet: Command-line Arguments

1. To make sure you understand the correct terminology, answer the following questions and then discuss your answers with two or three people sitting nearby.

- (a) Suppose you have a program named `prog.c`, what is the instruction you would type on the command line, to compile this program and create an executable named `prog`?

`gcc -o prog prog.c`

`gcc prog.c -o prog`

- (b) Now that you have an executable named `prog` assume that it is in your current working directory, give the command to run that executable with the command-line arguments `-k 3 myfile`.

`./prog -k 3 myfile`

- (c) Assume that the executable is in your parent directory, give the command to run the executable without any command-line arguments.

`../prog`

- (d) Assume you have changed back into the same directory as the executable. Give the command to run the executable where the resulting output is redirected to a file named `test1.out`.

`./prog > test1.out`

- (e) When you run the program, it interacts with the user expecting the user to type input. So far imagine that you've been doing this from the keyboard. Give the command to run the program and redirect the input so that the executable reads from the file `somefile.txt`.

`./prog < somefile.txt`

- (f) Put it all together. Show the command to run the executable `prog` with the command-line arguments `-k 3 myfile`, reading input from standard input redirected from `somefile.txt` and redirecting the output to `test1.out`.

`./prog -k 3 myfile < somefile.txt > test.out`

CSCB09H Worksheet: Command-line Arguments

2. Add to the following program so that it prints out the first two command-line arguments.

```
#include <stdio.h>

int main(int argc, char **argv) {

    printf("We have %d command-line arguments.\n", argc - 1);


    return 0;
}
```

3. Compile your program into an executable named **args_practice**.
gcc args_practice.c -o args_practice
4. Run your program with two command-line arguments. The first is your name and the second is your enthusiasm (between 0 and 5) for baseball. Run it with no command-line arguments or only one. What happens?
5. Change your program so that it takes your enthusiasm number (the second argument) and prints "Let's go Blue Jays!" that many times. Did you get an error when you ran the program? Remember that an individual command-line argument is a string (an array of char), so you need to use **atoi** or **strtol** to convert this argument into an integer.